F1 InsightX Project Context

Last updated: April 9, 2026

Overview

F1 InsightX is a production-minded Formula 1 analytics web application built as a polished full-stack portfolio project with real data infrastructure and a clear path toward strategy modeling, race intelligence, and fantasy optimization.

The project combines:

? a Next.js 16 frontend
? a Supabase-backed account and data layer
? a Python data platform
? FastF1-based session ingestion
? curated CSV and Supabase runtime data serving
? future-facing ML and strategy pipeline scaffolding

The current product surfaces are:

? Homepage with latest race context, race archive, constructors standings, and drivers standings
? Strategy Lab
? Fantasy Builder
? Account/Profile system
? Race detail pages
? Legal/privacy/cookie surfaces

Tech Stack

Frontend

? Next.js 16 App Router
? React 19
? TypeScript
? Tailwind v4 base integration with a heavily custom CSS system
? Recharts

Backend and APIs

? Next.js route handlers under ‘apps/web/src/app/api‘
? Supabase SSR and browser auth clients
? Supabase server/admin access for profile persistence and canonical data reads

Data Platform

? Python

? Jolpica reference data ingestion
? FastF1 ingestion pipeline for sessions and richer motorsport data
? curated CSV generation
? optional Supabase loading through ‘data/load_supabase.py‘

Deployment Target

? Vercel for frontend + route handlers
? Supabase Free / Postgres
? GitHub Actions for CI and scheduled data refresh

Current Product Architecture

Web App Structure

Primary app location:

? ‘apps/web‘

Important areas:

? ‘src/app‘ for pages and route handlers
? ‘src/components‘ for UI
? ‘src/lib‘ for auth, account, API helpers, server data access, security helpers, and UI/data utilities
? ‘public/assets‘ for team, driver, circuit, and logo assets

Data Architecture

There are currently two main runtime data paths:

1. Supabase-backed canonical data
2. curated CSV fallback for local or degraded runtime

This means the app can still run in a read-only mode when Supabase is not fully available, but the canonical direction is to keep refreshed curated data and optionally load that into Supabase.

Current Canonical Runtime Domains

? races
? circuits
? drivers
? constructors
? race results
? driver standings
? constructor standings
? race week context
? prediction snapshots

? fantasy inputs

Major Work Completed So Far

1. Core Next.js Product Foundation

The app was built into a multi-surface F1 product rather than a single landing page.

Delivered surfaces include:

? homepage
? race detail pages
? Strategy Lab
? Fantasy Builder
? account/profile area

The visual system uses a dark motorsport-inspired editorial design with:

? condensed display typography
? mono metadata
? layered gradients and atmospheric backgrounds
? team/driver visual identity assets

2. Data Ingestion and Canonical Data Layer

The original project already had curated CSVs and Supabase loading support.

That baseline was extended with a more production-minded FastF1 platform.

FastF1 Pipeline Added

A new Python pipeline was added under:

? ‘data_pipeline/‘

It includes:

? config
? ingest
? extract
? storage
? utils
? raw
? cache
? logs
? scripts

Main goals of the FastF1 layer:

? clean separation of ingestion from extraction and storage
? raw session archive layout

? rerunnable ingestion
? cache-first behavior
? support for single session, weekend, season, and full historical ingestion

FastF1 Scope

The ingestion system was built to pull:

? FP1
? FP2
? FP3
? Qualifying
? Sprint Shootout / Sprint Qualifying when available
? Sprint
? Race

Stored outputs include:

? session metadata
? results
? laps
? weather
? best lap summaries
? stint summaries
? optional telemetry/position outputs

FastF1 Storage Layout

Raw FastF1 outputs are stored consistently in:

? ‘data_pipeline/fastf1/raw/{year}/{round}_{event_name}/{session}/‘

Caching is isolated in:

? ‘data_pipeline/fastf1/cache/‘

Ingestion Runtime Behavior

The historical ingestion range was implemented to cover:

? 2020 through 2025 fully
? 2026 through the latest completed event available at runtime

This avoids hardcoding the end of 2026.

3. Canonical Curated Synchronization

The homepage and runtime app do not read raw FastF1 session folders directly.

To bridge the raw session archive into the canonical curated runtime layer, a sync step was added:

? ‘data/sync_fastf1_curated_sessions.py‘

This updates curated outputs such as:

? qualifying results
? race results
? strategy profiles
? driver standings
? constructor standings
? race week context
? prediction snapshots
? fantasy inputs

This made the app move from stale China 2026 context to Japan 2026 context once the underlying raw data existed.

4. Homepage Chronology and Standings Fixes

The homepage previously drifted because chronology and standings could lag behind newly ingested raw data.

The relevant server layer in:

? ‘apps/web/src/lib/server/f1-platform.ts‘

was hardened so the app now derives:

? latest completed race
? next race
? current standings snapshot

from canonical race/results data rather than trusting stale context rows alone.

That work also improved freshness selection between:

? Supabase-backed standings snapshots
? CSV fallback standings snapshots

The app now prefers the fresher ‘season + round‘ snapshot instead of blindly trusting whichever backend succeeded first.

5. Account and Profile System

The project includes a full account/profile flow built on Supabase auth and a ‘user_profiles‘ table.

Key profile capabilities now include:

? sign in / sign up
? Google OAuth support when enabled
? username generation
? username availability checks

? constructor selection
? driver selection
? avatar mode selection
? cooldown-based profile/username lock rules
? account export
? sign out

Account Routing

The product now treats ‘/account‘ as the canonical user-facing account/profile route.

‘/profile‘ was turned into a compatibility redirect to ‘/account‘ to avoid stale route links and 404s.

Profile Persistence Rules

Profile logic now supports:

? generated default usernames
? optional custom usernames
? 7-day username lock after a custom change
? 7-day constructor/theme lock after save
? favorite driver remaining editable during the profile lock window

6. Legal, Privacy, and Cookie Surfaces

The project was audited and upgraded with public-facing baseline legal and privacy surfaces.

Added pages:

? ‘/privacy‘
? ‘/terms‘
? ‘/cookies‘

Implemented:

? necessary-cookie disclosure
? privacy contact path support
? account data export
? cookie preference UI
? legal links in key UI areas

The current consent implementation is a browser-persisted preference flow intended as a launch baseline for a site that currently uses necessary auth/session cookies and no active marketing trackers.

7. Security Hardening Completed

Several meaningful security improvements were completed across the app.

Server / Client Boundary

Critical server-only modules were explicitly protected with ‘server-only‘:

? env access
? Supabase admin/server helpers
? profile server logic

Browser and Platform Security Headers

Baseline hardening headers were added at the proxy boundary, including:

? ‘X-Frame-Options‘
? ‘X-Content-Type-Options‘
? ‘Referrer-Policy‘
? ‘Permissions-Policy‘
? a restricted baseline CSP

Error and Cache Hardening

Public error and rate-limited responses were changed to use:

? ‘Cache-Control: no-store‘

This prevents shared caching of bad API responses.

Account/Auth Hardening

The following were tightened:

? sign-out same-origin verification
? reduced sign-up account enumeration
? reduced production client-side error disclosure
? safer auth callback handling
? genericized user-facing setup/config messages

Durable Rate Limiting Path

The app now supports two rate-limit modes:

1. shared/durable Upstash Redis REST mode
2. local in-memory fallback mode

Sensitive routes were wired to the shared-capable limiter, including:

? account profile reads and writes
? username check
? username suggest
? export
? auth callback
? sign-out

Username Surface Hardening

Username endpoints were tightened to:

? require same-origin verification
? require authentication
? ignore client-controlled exclusion IDs
? reduce attacker-facing validation leakage

Account Export Hardening

Account export was changed from a plain authenticated GET surface to:

? POST-based export
? same-origin verified
? non-cacheable

8. Repo and Public GitHub Readiness

The repository was hardened for public/private push readiness.

Completed improvements include:

? broader ‘.gitignore‘ coverage for env files
? confirmation that ‘.env.local‘ and related env files are not tracked
? ‘.env.example‘ kept as the safe placeholder file
? removal of unused default Next.js scaffold assets
? addition of ‘SECURITY.md‘
? README and deployment docs updated for the actual current stack

9. Branding and Assets

A logo asset structure now exists under:

? ‘apps/web/public/assets/logos/‘

The large-scale logo integration attempt was intentionally rolled back from the UI because it did not look right in the interface at that stage.

Current status:

? the logo assets exist in the repo
? the app is still using the prior visual identity approach in key UI areas

10. Homepage and Section Layout Refinement

Several homepage refinements were completed over time, especially around:

? constructors standings header
? drivers standings header
? spacing/padding alignment

? footer expansion
? cookie/legal footer integration

The current homepage includes:

? sticky nav
? hero
? constructors championship section
? products section
? race archive rail
? drivers championship section
? two-row footer with legal and product links

11. Current Security and Operational State

What is already in good shape

? no tracked ‘.env‘ secrets
? server-only service role usage
? route-level validation exists in key account flows
? public error/rate-limit responses are not cacheable
? privacy/legal surfaces exist
? repo now has ‘SECURITY.md‘
? durable rate-limiting path exists in code

What still depends on manual platform setup

? Upstash env vars for true shared rate limiting
? Supabase auth abuse controls
? CAPTCHA/provider restrictions if desired
? Vercel production env configuration
? privacy contact email in public env

12. Current Known Product Direction

The intended long-term product direction is:

? stronger Strategy Lab simulations
? race-week intelligence
? prediction modeling
? richer fantasy optimization
? telemetry-aware derived features

The current codebase is already structured to support those next phases without needing to start over architecturally.

13. Important Runtime Commands

Web app

```
npm install
npm run dev
npm run lint
npm run build
```

Core data pipeline

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install -r data/requirements.txt
```

Curated data refresh

```
python data/fetch_reference_data.py --start-season 2024 --end-season 2026
python data/normalize_results.py
python data/load_supabase.py
```

FastF1 pipeline

```
pip install -r data_pipeline/requirements.txt
python data_pipeline/scripts/run_fastf1_ingestion.py full-range
```

Sync FastF1 raw sessions into curated runtime outputs

```
python data/sync_fastf1_curated_sessions.py --season 2026
python data/load_supabase.py
```

14. Current Deployment Context

Target deployment stack:

? Vercel
? Supabase
? GitHub Actions

Important public envs:

? ‘NEXT_PUBLIC_APP_URL‘
? ‘NEXT_PUBLIC_PRIVACY_CONTACT_EMAIL‘
? ‘NEXT_PUBLIC_SUPABASE_URL‘
? ‘NEXT_PUBLIC_SUPABASE_ANON_KEY‘

Important server-only envs:

? ‘SUPABASE_SERVICE_ROLE_KEY‘
? ‘DATABASE_URL‘
? ‘UPSTASH_REDIS_REST_URL‘
? ‘UPSTASH_REDIS_REST_TOKEN‘

15. Current Repo State Summary

The repo now contains:

? the web app
? security hardening work
? legal/privacy/cookie surfaces
? account/profile system
? curated data pipeline
? FastF1 raw ingestion platform
? canonical sync bridge from FastF1 raw data to curated runtime data
? GitHub-ready repo hygiene and security policy

16. Guidance For Future ChatGPT Sessions

When using this project context in a future ChatGPT browser or upload workflow, the most important facts are:

1. ‘/account‘ is the canonical account/profile route.
2. The app uses Next.js App Router and Supabase.
3. Runtime data can come from Supabase or curated CSV fallback.
4. FastF1 raw ingestion exists under ‘data_pipeline/‘.
5. Curated runtime updates from FastF1 raw data happen through ‘data/sync_fastf1_curated_sessions.py‘.
6. Security hardening has already been done around headers, account routes, export flow, and rate limiting.
7. Shared rate limiting in production still depends on Upstash env configuration.
8. The project is production-minded but still evolving.

17. Recommended Context Tags

Useful labels for future assistant sessions:

? Next.js 16 App Router
? Supabase auth and profile persistence
? F1 analytics product
? FastF1 ingestion pipeline
? curated CSV fallback runtime
? Vercel deployment target
? production hardening completed in several stages
? legal/privacy baseline present